

Integrating Computer Vision: Agent-Based Crowd Simulation for Stampede Prevention and Event Safety

Project Review

Project Team:

Nivin P Louis (TL23BTAI0042)
Niranjana Sreevalsan (TL23BTAI0348)
Farhan Mohamed T P (TL23BTAI0308)
Nihal P Shakeer (TL23BTAI0249)

Project Guide:

Anju K P, Assistant Professor, Dept. AIML

Vidya Academy of Science and Technology, Thrissur



College Vision

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

College Mission

M1: Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.

M2: Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.

M3: Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Department Vision

To be a center of excellence in artificial intelligence and machine learning education, research, and innovation, nurturing competent professionals capable of developing intelligent, ethical, and sustainable solutions for societal and industrial advancement.

Department Mission

M1: Provide quality education in artificial intelligence and machine learning through effective teaching-learning practices, enabling students to acquire strong technical knowledge, analytical skills, and problem-solving abilities.

M2: Promote research, innovation, entrepreneurship, and interdisciplinary collaboration in emerging AI technologies to address industrial and societal challenges.

M3: Develop professionally competent graduates with ethical values, leadership qualities, lifelong learning abilities, and a commitment to societal well-being in a technology-driven world.

Index

- 1 Problem Statement
- 2 Motivation
- 3 Objectives
- 4 Literature Review
- 5 Architecture
- 6 Methodology
- 7 Implementation
- 8 Code Snippets
- 9 Challenges Faced
- 10 Future Scope
- 11 Reference

Problem Statement

Public venues generate dynamic pedestrian flows that static counts cannot describe. This project uses observed ATC trajectories and a ROS floor map to replay and simulate pedestrian movement, then checks short-horizon forecasts against held-out tracks.

- Counts and fixed capacity do not show individual routes or how movement changes over time.
- Real trajectories ground the simulator in observed speeds, paths, and starting states.
- Map-constrained agents provide a repeatable way to explore movement scenarios and compare forecasts.

- Observed pedestrian trajectories ground the simulator in measured paths and speeds.
- Individual agents make route and movement assumptions explicit.
- A mapped environment shows how pedestrian paths and density evolve over time.
- Replay and held-out comparisons let us inspect how simulation tracks observed movement.
- Repeatable simulation supports future study of event-safety scenarios under stated assumptions.

- **Prepare trajectories:** Clean ATC records, convert units, and summarize speed, routes, and endpoint zones.
- **Align the environment:** Use the ROS occupancy map to constrain simulated movement to walkable cells.
- **Initialize agents:** Start from observed snapshots or empirical movement profiles.
- **Simulate pedestrian flow:** Advance Social Force agents with map-based routing and obstacle rejection.
- **Evaluate forecasts:** Compare 1-, 5-, and 10-second predictions with held-out tracks and a constant-velocity baseline.
- **Visualize movement:** Inspect observed tracks, simulated flow, and forecast summaries in the replay dashboard.

Literature Review

- **Microscopic crowd models:** agent-based studies represent individual traits, goals, and interactions [3].
- **Observation-informed simulation:** research has combined crowd observations with agent-based models to forecast movement [4].
- **Evacuation and layout studies:** compare routes, exits, and scenarios using simulated agents [5].
- **Computer-vision studies:** estimate people, density, or flow from images, but require reliable camera-to-floor calibration for spatial simulation.

Literature takeaway

The supplied review spans computer vision, agent-based movement, and evacuation planning. The present implementation focuses on connecting real pedestrian trajectories to a mapped simulator.

Research Gap and Project Position

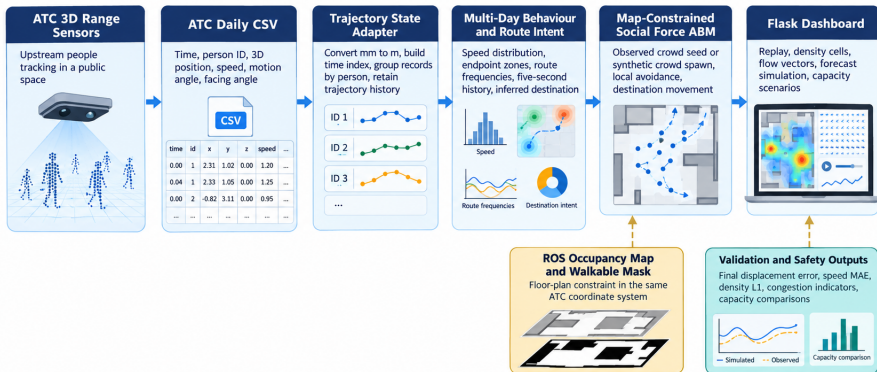
What prior work motivates

- Individual movement rules matter for crowd-level outcomes.
- Real observations can inform simulation state and behaviour.
- Layout and route changes can be studied through scenarios.

What this prototype contributes

- ATC trajectories aligned to the supplied ROS occupancy map.
- Empirical route and speed profiles used by Social Force agents.
- Held-out normal-flow forecast comparison in a replay dashboard.

System Architecture



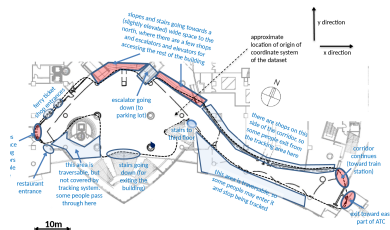
ATC Numerical Tracking State as CV-Equivalent Input

$s(i,t) = [\text{time}, \text{person_id}, \text{x_m}, \text{y_m}, \text{z_m}, \text{speed_m/s}, \text{motion_angle_rad}, \text{facing_angle_rad}]$

ATC provides tracked numerical trajectories, not raw camera images. A future computer-vision pipeline can connect to this architecture by producing the same map-aligned numerical state after detection, multi-object tracking, and floor-plane calibration.

Data and Environment

- ATC shopping-centre tracks from a 3D range-sensor system, Osaka, Japan [1, 2].
- 10 locally available daily files from Oct–Nov 2012.
- Each observation records time, ID, position, speed, motion angle, and facing angle.
- Positions use millimetres; speed uses millimetres per second.
- The ROS occupancy map uses the same coordinate frame [1].



Supplied ATC floor-plan slide

Data Preparation

- 1 Stream each daily CSV and build minute-level byte-offset indexes for interval replay.
- 2 Convert position and speed to metres and metres per second.
- 3 Filter trajectory jumps above 4 m and gaps outside 0.01–1.0 s.
- 4 Retain trajectories with at least 30 seconds and 20 samples.
- 5 Aggregate speed, heading, provisional endpoint zones, and frequent routes.

10-day data profile

307.7 million observations
165,089 filtered trajectories

340,953 tracked IDs
Mean speed: 0.839 m/s

Endpoint clusters are provisional analysis zones, not confirmed doors.

Agent-Based Simulation

Each pedestrian agent stores position, velocity, desired speed, facing direction, radius, destination, and arrival state.

Movement update

Agents follow a map-based shortest path toward a sampled destination. Nearby pedestrians contribute a distance-based repulsion, weighted by the agent's facing direction.

The model advances in 0.1 s steps, caps speed at 2.0 m/s, and rejects moves into non-walkable cells.

- **Observed start:** initialise agents from a selected crowd snapshot.
- **Synthetic start:** sample route, endpoint zone, and speed from empirical profiles.

Normal-flow calibration selected near-zero repulsion; dense-flow interaction behaviour remains unvalidated.

Forecast Evaluation

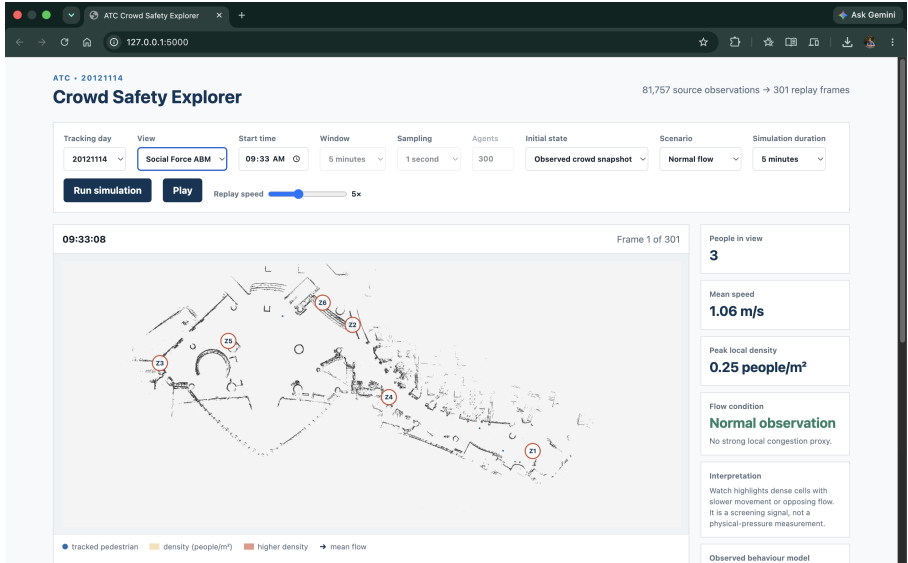
- 1 Calibrate parameters on three 10-second snapshots from 11 Nov 2012.
- 2 Hold out 14 Nov 2012; exclude that date from the calibration profile.
- 3 At six test times, seed the model from observations and five seconds of past history.
- 4 Compare the 1, 5, and 10-second forecasts with later observations for matching IDs.
- 5 Compare against constant velocity using the same starting observations.

Metrics

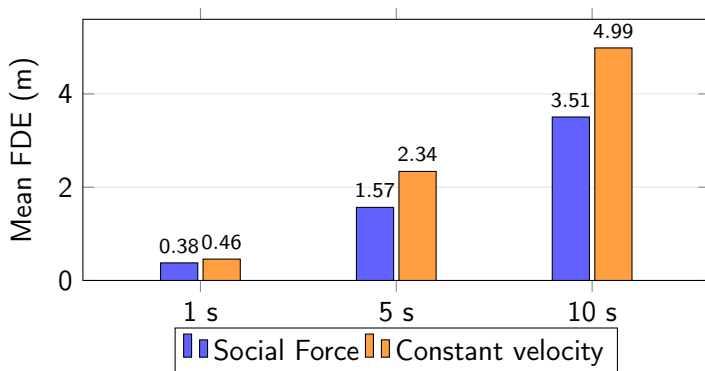
FDE: mean endpoint distance in metres. **Density L_1 :** difference in people-counts over 2 m cells, normalized by matched people.

Speed MAE: mean absolute speed error in m/s.

Dashboard Implementation



Implementation Results



FDE improved by 17.74%, 33.00%, and 29.71% at 1, 5, and 10 seconds. Six held-out snapshots per horizon on 14 Nov 2012; normal-flow trajectory forecasting only.

Held-Out Results

Horizon	FDE (SF / CV)	Density L_1 (SF / CV)	Speed (SF)	MAE
1 s	0.377 / 0.458 m	0.473 / 0.529	0.302 m/s	
5 s	1.567 / 2.338 m	1.032 / 1.441	0.452 m/s	
10 s	3.505 / 4.986 m	1.364 / 1.805	0.565 m/s	

Test coverage

All six requested snapshots had matched tracks. The 10-second results contain 77 matched person-snapshot pairs.

The experiment does not evaluate panic, physical pressure, stampede detection, or evacuation success.

Code Implementation: Track Cleaning

```
71 def main() -> None:
72     with SOURCE.open(newLine="") as handle:
73         reader = csv.reader(handle)
74         for row in reader:
75             if len(row) != 8:
76                 continue
77             try:
78                 stamp, person_id = float(row[0]), int(row[1])
79                 x, y, speed, heading = float(row[2]) / 1000, float(row[3]) / 1000, float(row[5]) / 1000, float(row[6])
80             except ValueError:
81                 continue
82             total_rows += 1
83             if 0 <= speed <= 4.0:
84                 speeds[speed_bin(speed)] += 1
85                 total_speed += speed
86                 total_speed_sq += speed * speed
87                 speed_count += 1
88             headings[heading_bin(heading)] += 1
89             current = people.get(person_id)
90             if current is None:
91                 people[person_id] = {"id": person_id, "first_t": stamp, "last_t": stamp, "first_x": x, "first_y": y, "last_x": x, "last_y": y}
92                 continue
93             delta_t = stamp - current["last_t"]
94             step = math.dist((x, y), (current["last_x"], current["last_y"]))
95             # Ignore discontinuities caused by a person ID being re-acquired.
96             if 0.01 <= delta_t <= 1.0 and step <= 4.0:
97                 current["path"] += step
98                 current.update({"last_t": stamp, "last_x": x, "last_y": y})
99                 current["samples"] += 1
100
101 valid = [record for record in people.values() if record["samples"] >= MIN_TRAJECTORY_SAMPLES and record["last_t"] - record["first_t"] >= 0]
102 starts = [(record["first_x"], record["first_y"]) for record in valid]
103 ends = [(record["last_x"], record["last_y"]) for record in valid]
```

Code Implementation: Social Force Model

```
257     nearby_indices = []
258     for row in range(cell_row - 1, cell_row + 2):
259         for col in range(cell_col - 1, cell_col + 2):
260             nearby_indices.extend(interaction_cells.get((row, col), []))
261     for other_index in nearby_indices[:48]:
262         if other_index == index:
263             continue
264         other = agents[other_index]
265         delta = position - other["position"]
266         distance = float(np.linalg.norm(delta))
267         if 0.001 < distance < 3.0:
268             unit = delta / distance
269             direction_to_other = -unit
270             ahead = max(0.0, float(np.dot(facing_vector, direction_to_other)))
271             # Agents retain rear collision avoidance but react more
272             # strongly to pedestrians within their forward view.
273             perception = 0.45 + 0.55 * ahead if self.facing_aware else 1.0
274             force += perception * self.repulsion_strength * math.exp(((agent["radius"] + other["radius"]) - distance) / s
275     new_velocity = velocity + force * dt
276     magnitude = float(np.linalg.norm(new_velocity))
277     if magnitude > 2.0:
278         new_velocity *= 2.0 / magnitude
279     proposal = position + new_velocity * dt
280     row, col = self.to_cell(proposal[0], proposal[1])
281     if not self.free[row, col]:
282         new_velocity *= 0.05
283         proposal = position
284     agent["velocity"], agent["position"] = new_velocity, proposal
285     if magnitude > 0.05:
286         target_facing = math.atan2(new_velocity[1], new_velocity[0])
287         turn = (target_facing - agent["facing"] + math.pi) % math.pi - math.pi
288         agent["facing"] += max(-0.35, min(0.35, turn))
289     target = self.zones[agent["destination"]]
290     if math.dist(proposal, (target["x"], target["y"])) < 0.75:
291         agent["arrived"] = True
292         agent["velocity"] = np.zeros(2)
```

Code Implementation: Forecast Evaluation

```
73     return sum(abs(pred_cells[key] - actual_cells[key]) for key in keys) / max(1, len(actual))
74
75
76 def evaluate_snapshot(simulator: SocialForceSimulation, dataset: Path, index: dict, timestamp: float, horizon: int, history_seconds: float)
77     initial = snapshot_by_id(dataset, index, timestamp, history_seconds)
78     if not initial:
79         return {"time": timestamp, "agents": 0, "matched": 0}
80     frames, _ = simulator.run(len(initial), horizon, "normal", list(initial.values()))
81     simulated = {person[0]: (person[1], person[2], person[3]) for person in frames[-1]["people"]}
82     actual = snapshot_by_id(dataset, index, timestamp + horizon)
83     shared = sorted(set(simulated) & set(actual) & set(initial))
84     if not shared:
85         return {"time": timestamp, "agents": len(initial), "matched": 0}
86     social_errors, baseline_errors, speed_errors = [], [], []
87     social_xy, baseline_xy, actual_shared = {}, {}, {}
88     for identifier in shared:
89         seed, truth = initial[identifier], actual[identifier]
90         sx, sy, simulated_speed = simulated[identifier]
91         bx = seed["x"] + seed["speed"] * math.cos(seed["heading"]) * horizon
92         by = seed["y"] + seed["speed"] * math.sin(seed["heading"]) * horizon
93         social_errors.append(math.dist((sx, sy), (truth["x"], truth["y"])))
94         baseline_errors.append(math.dist((bx, by), (truth["x"], truth["y"])))
95         speed_errors.append(abs(simulated_speed - truth["speed"]))
96         social_xy[identifier], baseline_xy[identifier], actual_shared[identifier] = (sx, sy), (bx, by), truth
97     return {
98         "time": timestamp,
99         "agents": len(initial),
100         "matched": len(shared),
101         "social_fde_m": float(np.mean(social_errors)),
102         "constant_velocity_fde_m": float(np.mean(baseline_errors)),
103         "social_speed_mae_m_s": float(np.mean(speed_errors)),
104         "social_density_l1": density_l1(social_xy, actual_shared)
```

Challenges Faced

- **Large source files:** streaming reads and minute-level indexes support practical replay.
- **Map ambiguity:** the ROS map includes broad unknown/free areas; a trajectory-derived walkable mask constrains agents.
- **Uncertain destinations:** route intent is inferred from short history, heading, and other-day route frequencies.
- **Limited emergency evidence:** calibration and test data describe ordinary shopping-centre flow, not crowd-crush events.

Main limitation

The current metrics support short-horizon normal-flow forecasting only. They do not establish a stampede warning capability.

- ❶ Integrate calibrated computer vision to turn video observations into map-aligned tracks.
- ❷ Expand evaluation to more dates, venues, crowd densities, and independent datasets.
- ❸ Calibrate and validate dense-flow and emergency behaviour with scenario evidence and expert-defined measures.
- ❹ Compare exit, route, and staffing interventions using evacuation time, peak density, and uncertainty.

Long-term direction

Develop the simulator as a decision-support tool that tests safety scenarios; treat its forecasts as model-based estimates, not guaranteed predictions.

- ❶ ATR Interaction Science Laboratories. *ATC shopping-centre tracking dataset and occupancy map*.
https://dil.atr.jp/ISL/crest2010_HRI/ATC_dataset/
- ❷ Brščić, D., Kanda, T., Ikeda, T., and Miyashita, T. (2013). Person position and body direction tracking in large public spaces using 3-D range sensors. *IEEE Transactions on Human-Machine Systems*, 43(6), 522–534.
- ❸ Khan, S. and Deng, Z. (2024). Agent-based crowd simulation: an in-depth survey of determining factors for heterogeneous behaviour. *The Visual Computer*, 40, 4993–5004.
<https://doi.org/10.1007/s00371-024-03503-2>
- ❹ Makinoshima, F. and Oishi, Y. (2022). Crowd flow forecasting via agent-based simulations with sequential latent parameter estimation from aggregate observation. *Scientific Reports*, 12, 11168.
<https://doi.org/10.1038/s41598-022-14646-4>
- ❺ Alidmat, O. et al. (2025). Simulation of crowd evacuation in asymmetrical exit layout based on improved dynamic parameters model. *IEEE Access*, 13, 7512–7525.
<https://doi.org/10.1109/ACCESS.2024.3420068>
- ❻ Helbing, D. and Molnár, P. (1995). Social force model for pedestrian dynamics. *Physical Review E*, 51, 4282–4286.
<https://doi.org/10.1103/PhysRevE.51.4282>